Micromouse Documentation

McMaster Micromouse

Version 0.1 (Draft)
Last Updated: August 12, 2026

Contents

Chapter 1

Introduction

1.1 What is Micromouse?

A Micromouse is a small autonomous robot that navigates and solves a maze as quickly as possible. The robot begins with no knowledge of the maze layout and must explore it using external sensors. As it explores, it constructs an internal map of the maze, determines the most efficient route, and then completes subsequent runs at increasingly high speeds.

Micromouse has existed as an international engineering competition hosted by IEEE for decades, and remains a great project for learning embedded systems. A successful Micromouse requires the integration of mechanical design, electronics, and software development.

1.2 Club Objectives

The goal of our club is to make the process of designing, building, and programming a Micromouse accessible to students of all experience levels. Robotics projects often require knowledge from multiple disciplines, which can make them difficult to approach independently. Through structured workshops, hands-on resources, and comprehensive documentation, the club aims to lower this barrier and provide a clear path from beginner to autonomous robot.

Participants will be guided through the complete development process, from assembling hardware and programming embedded systems to implementing sensing, localization, and path-planning algorithms. Rather than simply providing a finished design, the club emphasizes understanding the engineering principles behind each subsystem so members can modify, improve, and extend their own robots.

While the club provides a complete reference robot, members are encouraged to explore their own ideas and improvements. Topics such as custom PCB design, alternative hardware, and advanced control strategies are beyond the scope of the workshops but are left open for exploration. The provided platform should be viewed as a foundation upon which participants can experiment, iterate, and develop their own designs.

1.3 How to Use This Guide

This document serves as both a workshop and technical reference for the Micromouse Club. It is intended to complement our workshops by providing detailed explanation of concepts, design decisions, and implementation details that we may not have time to cover during workshops. It also allows members to revisit or look ahead.

The guide is organized into four main sections:

1. **Hardware:** Introduces the electrical components used in the robot. Explains how they operate and covers wiring.
2. **Software:** Describes sensor integration, motion control, localization, and path- finding algorithms.
3. **Reference Implementation:** Documents the club's reference Micromouse design. This section serves as a starting point for members building their own robots.

Chapter 2

System Overview

2.1 Robot Architecture

In general, a micromouse is a closed-loop system where the bot is constantly **sensing**, **processing**, and **acting**. The bot will autonomously adjust itself while going through the maze.

- **Sensing** comes from the distance sensors and wheel encoders. They feed data into the system to be processed.
- **Processing** is done in the microcontroller, where it combines sensor data and maze-solving algorithms to determine motor movement.
- **Acting** from the wheels results from the determined motor commands.
- **Power** is also required to feed voltages/currents to each subsystem. This is done though a battery and any power regulators.

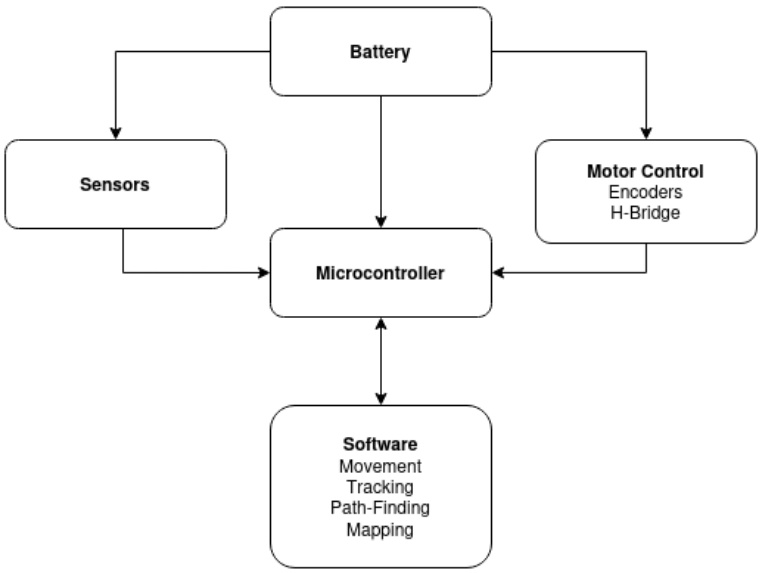


Figure 2.1: General Block Diagram

2.2 Hardware Overview

2.3 Software Overview

Part I

Hardware

Chapter 3

Materials

3.1 Components

There are a variety of components that can be used for a Micromouse. The main categories are summarized in System Overview. Table 3.1 lists all the components provided in our kits.

Components	**Quantity**	**Purpose**
ESP32-DEVKIT	1	Selected microcontroller. Main "brain" of the bot.
VL53L1X	3	Time of Flight sensor. Used to align the bot.
TB6612FNG	1	Motor driver; to control the motors.
G12-N20 Motors	2	Motors for the wheels.
N20 Motor Encoders	2	Attached to motors to track of motor position.
N20 Gear Wheels	2	Wheels for the bot to move.
Battery (WIP)	1	Power Supply

Table 3.1: Component List

These provide a good starting point for a basic Micromouse bot, and will be the parts we focus on throughout our workshops. The Hardware Theory section will also go in-depth on how each component works. However, you are free to choose your own parts. Some alternative options are described in the next section.

3.2 Alternatives

This section briefly introduces several alternative components that can be used for your own Micromouse, along with the rationale behind our selected components. Keep in mind that the options presented are not exhaustive; you are encouraged to research additional alternatives. A summary list (Table 3.2) is included at the end of this section.

Power Supply
The battery must provide sufficient voltage for the motor drivers while supplying enough current during acceleration and sudden load changes. The ESP32 and sensors use relatively low power, but the motors require several hundred milliamps during regular operation, and even more during startup.

For testing, 9V batteries can be used, but are not recommended for normal operation due to their limited current; it cannot run the bot for a long time. A better beginner option would be AA NiMH battery packs (5 or 6 for approximately 6V or 7.2V) as they provide more current than 9V batteries. Alkaline batteries are also acceptable for basic testing, but are generally less suited for prolonged operation. However, battery packs in general are much larger and heavier than lithium batteries.

Lithium batteries, such as Li-ion and LiPo, are the recommended batteries to use due to their higher energy density. However, they require additional care during use. Lithium batteries require dedicated chargers; overcharging, over-discharging, short circuits, or physical damage can damage the battery and may create safety hazards. As a result, traditional battery packs may be a simpler and safer option while learning how to design and operate the bot.

2×18650 Li-ion cells are a good middle ground as they provide the high capacity and current. 2S LiPo batteries are recommended for more advanced bots as they are lightweight and compact. However, they require a balance charger, should not be discharged below their rated voltage, and should be stored and handled carefully.

Microcontroller
Micromouse robots require enough of computing power, memory, and peripheral support to handle sensor readings, motor control, and path-finding algorithms. While basic Arduino boards such as the Arduino Uno or Nano(ATmega328P) can be used for simple robots, they become restrictive for more advanced algorithms.

The STM32 family (e.g. the STM32F103, 'blue pill, 'black pill', and STM32F4 Series) is a popular choice for more advanced Micromouse bots due to its performance, low-level hardware control, and extensive timer and interrupt capablities. However, due to the learning curve of using the STM32CubeIDE, can be more challenging for beginners. For this reason, an ESP32 board was chosen to provide a more accessible development while still providing sufficient performance for a Micromouse.

Alternative ESP32 boards, such as the ESP32-C3 and ESP32-S3, can also be considered. These boards are smaller and have different peripheral options, but have fewer available GPIO pins compared to some ESP32 development boards. With careful planning, they can still support the required sensors, motor drivers, and encoders.

Distance Sensors
Distance sensors are one of the most important parts of a Micromouse bot. When selecting a sensor, it is important to consider range, update rate, field of view, and physical size.

We will be using the VL53L1X time-of-flight (ToF) sensor due to its relatively long sensing range, accuracy, and size. Alternatives of the ToF sensor include the VL53L0X (shorter range) or VL53L4CX (longer range).

Infrared (IR) distance sensors are another common choice, especially in advanced Micromouses. Compared to ToF sensors, IR sensors have faster update rates and lower latency. However, they require analog signal processing and require more calibration.

Ultrasonic sensors may also be used for slower bots, but are not recommended in general. They are physically larger and are slower since they use a sound waves rather than light.

IMU Sensors
For advanced and faster Micromouses, it is worth considering adding an IMU (inertial measuring unit). Common choices include the MPU6050 and MPU6500. These provide more accurate turns and smoother motion control, but are not strictly necessary for beginner robots. As a result, we have not provided any in the kits as beginner robots work with just wheel encoders.

Motors
Micromouse motors do not need huge amounts of power. The focus is on speed, control, smoothness, size, and accurate encoder readings. Encoders often come together with the motor.

N20 DC Gear Motor would work are common choices for beginner Micromous bots. They come in various gear ratios, are compact, cheap and widely available. The ones we provide have a gear ratio of 30:1, however other ratios can also be used. Coreless DC gear motors are also worth looking into for advanced Micromouses.

Stepper motors are not recommended for Micromouse bots. Although they provide precise position control, they are larger, heavier, consume more power, and have lower speeds and acceleration compared to DC motors.

Motor Driver
The motor driver acts as the bridge between the microcontroller and the motors. Since we are just using N20 gear motors, the chosen motor driver was the TB6612FNG. It is a popular choice for small bots as it is small, cheap and can control two separate motors.

Alternatives include the DRV8833 (similar to TB6612FNG) or the DRV8871 (for larger motors). The L298N is not really recommended due to its bulky size and large voltage drops.

Summary

Components	**Selected**	**Alternative**
Microcontroller	ESP32-WROOM	ESP32-C3, ESP32-S3, STM32F103/F4
Distance Sensor	VL53L1X	VL53L0X, VL53L4CX, IR sensors
IMU	None	MPU6050, MPU6500
Motors	N20 DC Gear Motor(30:1)	Other N20 gear ratios, Coreless DC motors
Motor Driver	TB6612FNG	DRV8833, DRV8871
Battery	TBD	AA NiMH, 18650 Li-ion, 2S LiPo

Table 3.2: Summary of Component Alternatives

Chapter 4

Hardware Theory

Rather than a complete explanation of how each component works in detail, this section just highlights necessary information for the Micromouse build. Some additional references or further readings may be included thoughout.

4.1 ESP32

We will be using the 30-pin ESP32 DevKit. Although it is commonly known for its Wifi and Bluetooth capablities, it also has strong processing power, large memory capacity and is beginner-friendly.

Some key specs are:

- 32-bit dual-core processor (Up to 240MHz)
- 520KB SRAM
- 4MB Flash
- Up to 25 usable GPIO pins

Since all the wiring is done to the ESP32, this section will elaborate on the pinout of the ESP32. It is mainly based on the article ESP32 Pinout Reference. Only the relevant sections are highlighted here.

For our bot, we will be using the GPIO pins for general-purposes, in addition to PWM and I2C pins. Figure 4.1 provides a visual summary of which peripherals are available for each pin.

There are 25 GPIO pins available for general use. GPIO 35-36 are input only pins that do not include internal pull-ups or pull-downs; an external resistor can be added if needed. Pay attention to pins GPIO 0, 2, 5, 12, and 15 as they are connected to boot configuration(strapping) and may prevent the ESP32 from starting; try not to use them unless necessary.

PWM (Pulse Width Modulation) pins are used to control motor speed. Instead of producing a constant analog voltage, PWM constantly switches between HIGH and LOW by adjusting the "duty cycle" (percent of time the pin is ON vs OFF). This allows the average voltage supplied to the

motor to be controlled. Any output GPIO pin can be used for PWM.

I2C pins are the SDA (data line) and SCL (clock line) pin. By default, GPIO21(SDA) and GPIO22 (SCL) are used as the I2C pins, but any GPIO pin can be configured as SDA and SCL. Multiple devices can share the same I2C bus as long as each device has a unique device. Since we are using three ToF sensors, the XSHUT pins are used to configure their unique addresses. (See Time-of-Flight Sensors)

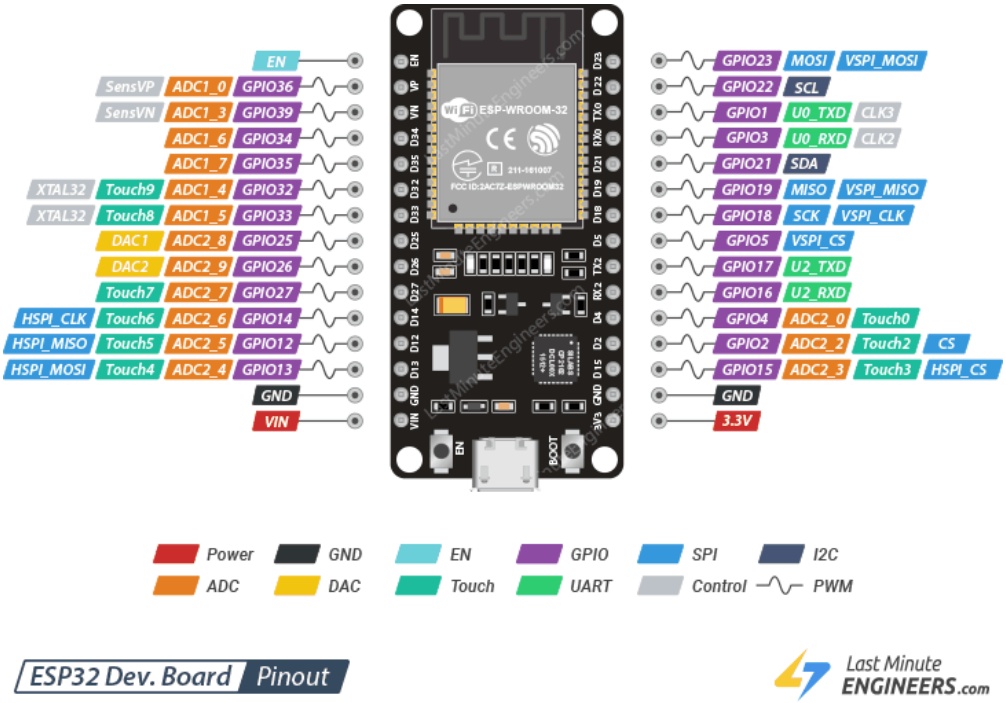


Figure 4.1: ESP32-Pinout

The ESP32 also includes interrupts, which allow the board to quickly respond to external events without constantly checking the state of a pin (polling). For Micromouse, interrupts are used for reading motor encoder data; this allows accurate measurement even while the processor is performing other tasks. (See Encoders)

Finally, the ESP32 runs at 3.3V. It has two power pins: VIN and 3V3. The VIN pin can accept external power (typically 5V to 12V) which is internally regulated to 3.3V. The 3V3 pin provides 3.3V to compatible components. DO NOT apply voltages higher than 3.3V to the 3V3 pin or GPIO pins. Avoid powering the ESP32 simultaneously through both the USB and external power source. All connected components must share a common ground (GND).

4.2 Time-of-Flight Sensors

The VL53L1X is a time-of-flight (ToF) laser-ranging sensor with an accurate ranging up to 4m (under ideal conditions) and supports measurement rates up to 50Hz. Figure 4.2 shows the module used. Download the VL53L1X datasheet for more full details.

Figure 4.2: VL53L1X Module

Wiring
The module has the following pins:

- **VCC:** Power input (3V to 5V).
- **SDA:** I2C data line.
- **SCL:** I2C clock line.
- **GND:** Common ground for power and logic.
- **XSHUT:** Shutdown control pin; pulling this to LOW puts the sensor into standby. Also used to program custom I2C address.
- **INT:** Interrupt output pin. Not necessary if polling is used.

VCC and GND are the power pins. VCC should be connected to 3V3 from the ESP32, while GND should be connected to the common ground shared by all components.

I2C (Inter-Integrated Circuit) is a type of communication protocol that supports multiple devices on one bus. It is synchronous and requires two wires: one line for data signal, and one for the clock (synchronization and timing control for data transmission). Each device is identified by a unique address. The default address is 0x29.
(I2C - Sparkfun Electronics)

Since we will be using three sensors, the default address cannot be used since the microcontroller will be unable to determine which sensor the data belongs to. The XSHUT pin can be used to set separate address for each sensor; it must be connected to a distinct GPIO pin on the microcontroller. Once all the sensors have a new address set, they can communicate on the same SDA and SCL line.

Although the VL53L1X provides an interrupt pin (INT), this guide uses polling for simplicity as it is sufficient for the update rates required by the bot. Interrupts will only be used for the encoders.

How it Works / Features
The VL53L1X detects distance by emitting an invisible 940nm laser and measures the time it takes for the reflected light to return to the sensor.
Main sensor data includes:

- Ranging distance(mm)
- Return signal rate
- Ambient signal rate (amount of background light detected)
- Range status (indicates if a measurement is valid)

It typically has a full field of view (FoV) of 27°. As the distance to an object increases, the sensing area also becomes larger. Distant measurements may be influenced by nearby walls or corners and can be less accurate than close-range readings.

Chapter 3 of the VL53L1X datasheet provides more detail on customizable parameters for the sensor. This includes region of interest (ROI), distance mode, and timing budget. See Reading ToF Sensors for how to adjust everything in code.

4.3 Motors

N20 geared motors are small DC motors combined with a gearbox, allowing for high torque within a limited space. "N20" refers to the size of the motors, which are usually 15mm long, 12mm wide, and 10mm high.

Figure 4.3: GA12-N20 DC Gear Motor

The motors come in a variety of gear ratios; we have chosen a ratio of 30:1 as a good middle ground. In general, a lower gear ratio (e.g. 10:1) increases speed, while a higher gear ratio (e.g. 100:1) increases power by providing more torque.

To control them, a (Motor Driver) must be used between the microcontroller and motors. Since motors require more power than microcontrollers (which work on low voltage and current), they cannot be driven directly from the ESP32. Encoders are also included with the motors given, allowing the ESP32 to measure wheel rotation.

4.4 Motor Driver

Motor drivers act as the bridge between the microcontroller and motors by amplifying the low power signals from the microcontroller. This allows the microcontroller to safely control higher voltages and currents while independently control each motor's speed and direction.

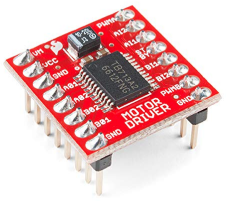


Figure 4.4: TB6612FNG Motor Driver

The TB6612FNG is a dual-bridge H-bridge, where the H-bridge consists of four switches in an H-shape which allow the motors to move in either directions (ccw and cw).

Wiring

- **VM:** Power supply for motors (15V Max)
- **VCC:** Power supply for logic (2.7V to 5.5V)
- **GND:** Common ground
- **AIN1/AIN2:** Direction control for motor A
- **BIN1/BIN2:** Direction control for motor B
- **PWMA/PWMB** PWM speed for motor A and B
- **STBY:** Standby (Must be pulled HIGH to enable the driver)
- **A01/A02:** Output for motor A
- **B01/B02** Output for motor B

Notice that there are two power inputs: one for the motors and one for the control logic. This allows the motors to be powered directly from the battery, while having the control logic powered by the ESP32. Separating the two power inputs help isolate electrical noise generated by the motors from the microcontroller.

The STBY (standby) pin enables or disables the motor driver. When LOW, both H-bridges are disabled so the motors cannot move. This pin can be connected to a GPIO if control is desired, or it can be tied to HIGH (connect to 3V3) if the driver should always remain enabled.

4.5 Encoders

The motors given have magnetic encoders attached to them. These are adapted rotary encoders that measure the rotation of the motor, allowing a way to keep track of how far and fast the motors are going. However, since the encoders are attached to the gearboxes of the motors, the encoders rotate faster than the wheel; for our 30:1 gear ratio, the motor shaft rotates 30 times for every one rotation of the wheel. The encoder measurements must be converted to account for the gears.

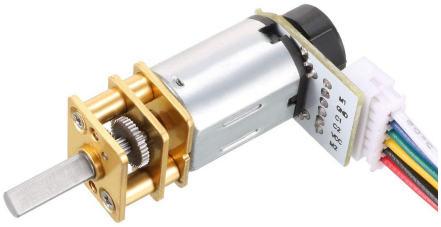


Figure 4.5: N20 Gear Motor with Encoder

The sensor is made of a center magnet and two halleffect sensors (figure 4.5). As the shaft spins, the halleffect sensor toggle between high and low states, producing a square wave. With one hall sensor (single channel), the speed and number of rotations of the motor shaft can be measured.

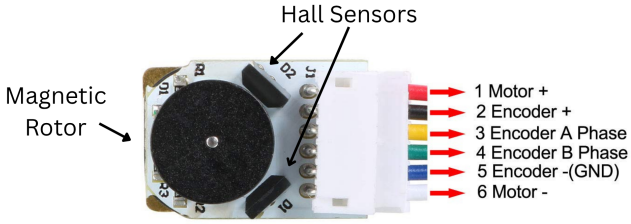


Figure 4.6: Encoder

Since using a single hall sensor only produces one square wave, a rotation of the wheel can be calculated with the number of rises that occur on the graphs. The encoders we are using have a 7 PPR (pulses per revolution), meaning there will be 7 rises for each 360°. To calculate the total PPR for our specific gear ratio:

$$Output\ Shaft\ PPR = (Encoder\ PPR)(Gear\ Ratio)$$

$$Output\ Shaft\ PPR = (7)(30) = 210\ pulses/rev$$

With two hall sensors being used, the CPR (counts per revolution) is 4 times the PPR. The encoder reads both rising and falling edges, making the final CPR for our 30:1 motor 210(4) = 840 pulses/rev.

The two hall sensors (quadrature encoder) allow the direction of rotation to be found; the sensors produce two square waves that are offset by 90°, where the order of signals determine the direction (figure 4.7, 4.8). Encoder pin A leads encoder pin B when the motor rotates in a clockwise direction, and vice versa for counterclockwise. This means direction is determined by comparing the values of encoder pin A and B.

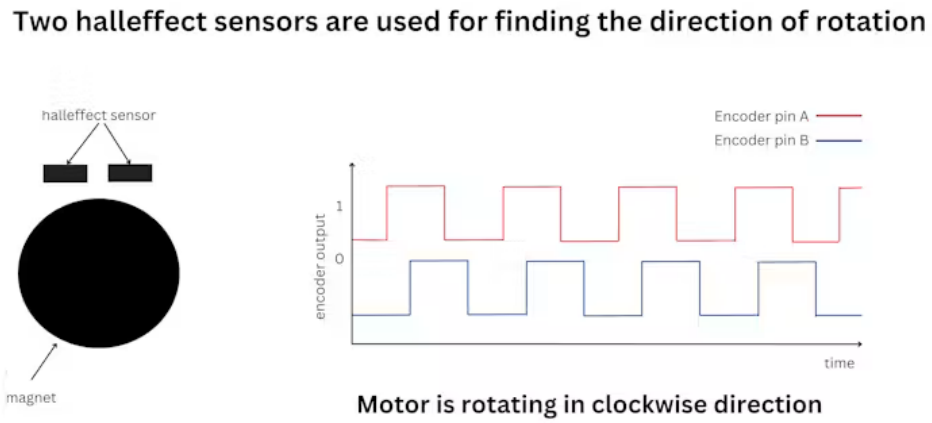


Figure 4.7: Clockwise Direction

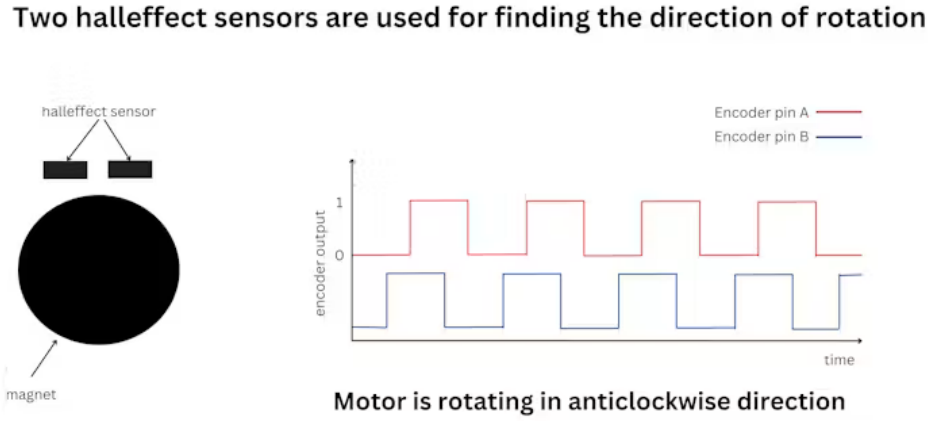


Figure 4.8: Counterclockwise Direction

(Graphs from "Reading the encoder value of N20 motor using ESP32")

Wiring
The wire labels are shown in figure 4.6.

M1 and M2 (red and white) are the two motor connections that should be connected to the motor driver (A01/A02/B01/BO2). The polarity doesn't matter; swapping the two wires swap the rotational direction of the motors (ccw/cw). However, this can also be adjusted in the code.

The black and blue wires(2 and 4 on the figure) are the VCC and GND pins. VCC (black) should be connected to 3V3, while GND should be connected to the same shared GND throughout the board.

C1(Encoder A phase) and C2(Encoder B phase) are the yellow and green wires that connect to a GPIO on the microcontroller. C1 should be connected to an interrupt pin, and both C1 and C2 need to have pull-up resistors. Ensure the chosen pin have internal pull-ups, or add your own external pull-up resistor if the pin does not have one. Since our motor driver can handle direction, we only need to use C1 from our encoder.

Chapter 5

Electrical Design

This section provides the wiring we used for the example bot. Keep in mind that many of these pin connections can be changed. Refer to the corresponding sections in Hardware Theory for explanations behind each pin connection and alternative wiring.

5.1 Wiring Schematic

NOTE: The ToF sensor in the diagram is not the same one we are using. However, they share the same pins. A table of pin assignments to the ESP32 can be found in Pin Assignments.

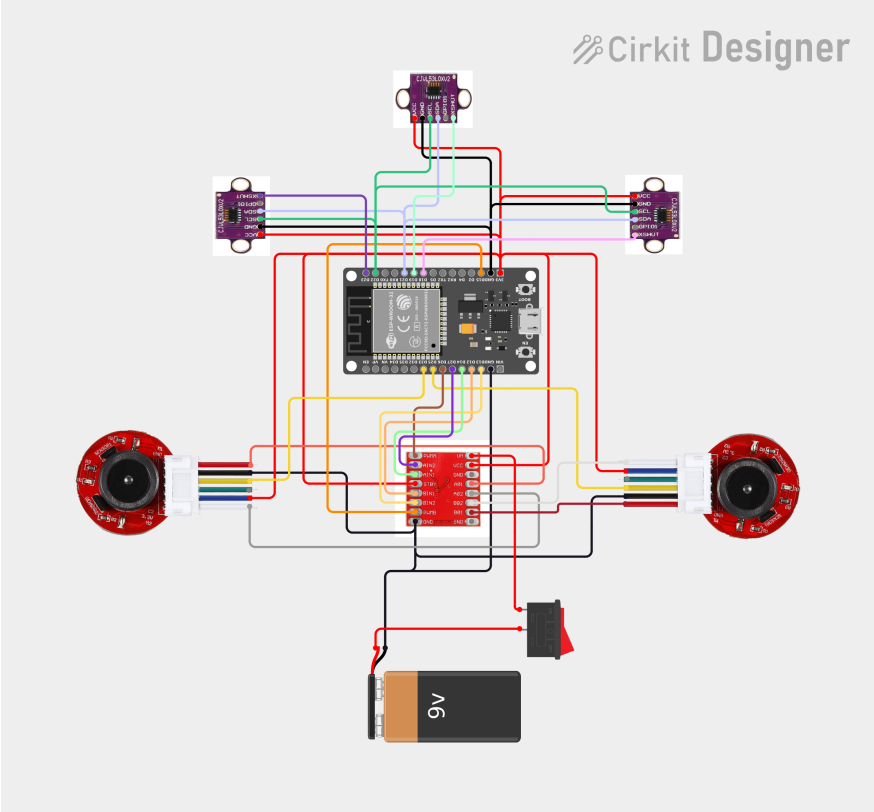
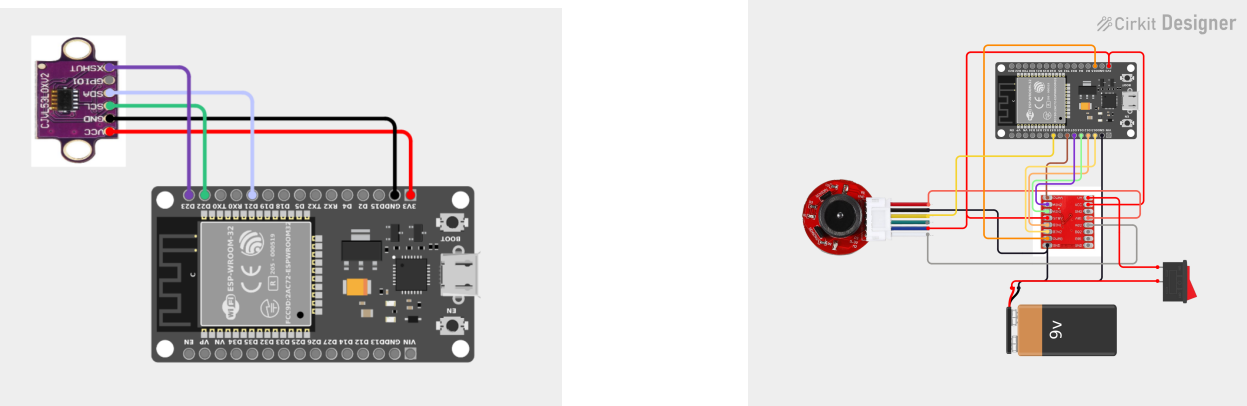


Figure 5.1: Full Wiring Diagram



(a) ToF Wiring

(b) Motor Driver and Encoder Wiring

Figure 5.2: Additional Circuit Figures

5.2 Pin Assignments

ESP32 Pin	**Connected**
VIN	Battery (+)
3V3	ToF VCC, TB6612FNG VCC & STBY, Encoders VCC (Blue)
GND	All GNDs
GPIO 21	All ToF Sensors (SDA)
GPIO 22	All ToF Sensors (SCK)
GPIO 23, 18, 19	ToF XSHUT
GPIO 26	TB6612FNG PWMA
GPIO 15	TB6612FNG PWMB
GPIO 27	TB6612FNG AIN1
GPIO 14	TB6612FNG AIN2
GPIO 12	TB6612FNG BIN1
GPIO 13	TB6612FNG BIN2
GPIO 25, 33	Encoder Channel A (Yellow)

Table 5.1: ESP32 Pin Assignments

Part II

Software

Chapter 6

Software Architecture

6.1 Project Structure

6.2 Control Loop

Chapter 7

Firmware Setup

7.1 Development Environment

To upload code onto the ESP32, you can use either Arduino IDE or PlatformIO on VSCode. This section will go through downloading and setting up the development environments.

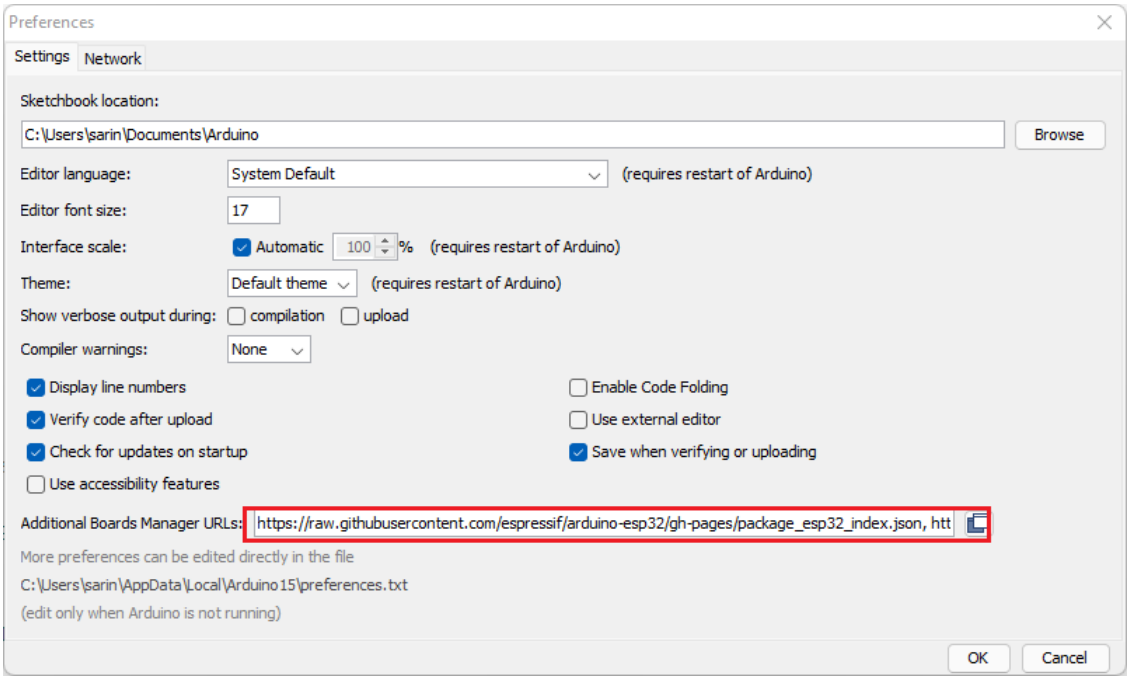
Arduino IDE
Arduino IDE can be downloaded on their website: https://www.arduino.cc/en/software/

Figure 7.1: Arduino IDE Download

Once the Arduino IDE is installed, the ESP32 boards need to be added on as add-ons. The following instructions are taken from "Installing the ESP32 Board in Arduino IDE"

1. Go to **Files** > **Preferences** in Arduino IDE
2. Enter the following into "Additional Board Manager URLs" field:

   ```
   https://raw.githubusercontent.com/espressif/arduino-esp32/
   gh-pages/package_esp32_index.json
   ```



3. Go to **Tools** > **Board** > **Boards Manager...**
4. Search for **ESP32** and install "**ESP32 by Expressif Systems**"

PlatformIO IDE
PlatformIO can be added to VSCode by searching "PlatformIO IDE" in extension. (platformio.org/ide?install=vscode)

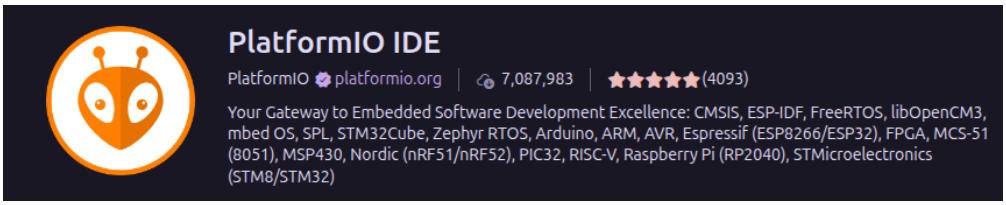


Figure 7.2: PlatformIO IDE

7.2 Libraries

VL53L1X Arduino Library

7.3 Uploading Code

Chapter 8

Sensors

This chapter will provide code examples on how to use the sensors. Look at the Libraries section and Hardware Theory for more background details and explanations. All the example code can also be found in our simulator.

8.1 Reading ToF Sensors

The general steps for acquiring ToF sensor data are:

1. Import <Wire.h> and <VL53L1X> and define pins for I2C wires and XSHUT
2. Create an object for each sensor to address and read them independently later
3. Choose a unique I2C address to reassign each sensor to
4. in setup():
    - Start serial (for debug output) and I2C bus
    - RESET all sensors (XSHUT LOW) so none of them are activated on the bus yet
    - Reassign a new address to each sensor one at a time
5. In loop(), call each sensor's read function to obtain and do something with the acquired distance values

VL53L1X libraries and variables

```
//import libraries
#include <Wire.h>
#include <VL53L1X.h>

// Pin definitions
#define SDA 21
#define SCL 22

#define XSHUT_FRONT 19
#define XSHUT_LEFT 18
#define XSHUT_RIGHT 23

// I2C addresses (default is 0x29 for all)
#define ADDR_FRONT 0x30
#define ADDR_LEFT 0x31
#define ADDR_RIGHT 0x32

VL53L1X sensorFront;
VL53L1X sensorLeft;
VL53L1X sensorRight;
```

A helper function can be created to initialize each ToF sensor. Each sensor needs to be assigned a different address. If initializing fails, the output in the serial monitor will display which sensor failed to initialize.

```
void initToF(VL53L1X& sensor, int xshutPin, uint8_t newAddress, const
    char* name) {
  digitalWrite(xshutPin, HIGH); // enable sensor
  delay(10);

  sensor.setTimeout(500);

  // intialize
  if (!sensor.init()) {
    Serial.print("Failed to boot VL53L1X: ");
    Serial.println(name);
    while (1) {
      delay(100);
    }
  }
  sensor.setAddress(newAddress);
  sensor.startContinuous(50);  // 50ms between measurements
}
```

In the setup, we want to open up the serial port, set up I2C communication, and assign the different addresses for our multiple ToF sensors.

```
void setup() {
  Serial.begin(115200);
  while (!Serial) { delay(1); }
  delay(1000);  // give power a moment to stabilize

  Wire.begin(SDA, SCL);   // start I2C bus
  Wire.setClock(400000);  // faster I2C speeds (400kHz)

  // hold all sensors in reset first (off the bus)
  pinMode(XSHUT_FRONT, OUTPUT);
  pinMode(XSHUT_LEFT, OUTPUT);
  pinMode(XSHUT_RIGHT, OUTPUT);

  digitalWrite(XSHUT_FRONT, LOW);
  digitalWrite(XSHUT_LEFT, LOW);
  digitalWrite(XSHUT_RIGHT, LOW);

  delay(10);

  // assign each sensor a unique address ONE AT A TIME
  initToF(sensorFront, XSHUT_FRONT, ADDR_FRONT, "FRONT");
  initToF(sensorLeft, XSHUT_LEFT, ADDR_LEFT, "LEFT");
  initToF(sensorRight, XSHUT_RIGHT, ADDR_RIGHT, "RIGHT");
}
```

In the loop, we can actually gather the sensor data by using .read(). This will print out the values of each sensor in the serial monitor.

```
void loop() {
  Serial.print("ToF Front: ");
  Serial.print(sensorFront.read());
  Serial.print(" mm  Left: ");
  Serial.print(sensorLeft.read());
  Serial.print(" mm  Right: ");
  Serial.print(sensorRight.read());
  Serial.println(" mm");

  delay(100);
}
```

8.2 Reading Encoders

The following examples will be for one motor/encoder. The second encoder should act the same way.
Encoder and motor pin definitions. A volatile variable is used to indicate to the compiler that the variable can change at anytime. This prevents it from being stored in cache. A volatile variable should always be used for any variable shared between the ISR (Interrupt Service Routine) and main code.

```
// Encoder
#define AIN1 14
#define AIN2 27
#define PWMA 26
#define ENCODERA_A 25

// PWM Config
#define PWM_FREQ 20000
#define PWM_RES 8
#define PWM_CHA 0

volatile long encoderCountA = 0;
```

Interrupt functions on the ESP32 must be marked as IRAM_ATTR. This is a function that places the function in IRAM (internal RAM) instead of the default cache, allowing the CPU to access the functions regardless of what is happening in the flash cache (which may sometimes temporarily disable certain operations).

```
void IRAM_ATTR readEncoderA() {
  encoderCountA++;
}
```

To setup the encoders, define the pin modes and attach the interupt. The attachedInterrupt function can have three modes for the encoder; RISING, FALLING, and CHANGE. These configure which edge triggers the interrupt (LOW → HIGH, HIGH → LOW, or both).

```
void setup() {
  Serial.begin(115200);

  pinMode(ENCODERA_A, INPUT_PULLUP);

  // attachInterrupt(digitalPinToInterrupt(PIN), ISR FUNCTION, MODE)
  attachInterrupt(digitalPinToInterrupt(ENCODERA_A), readEncoderA,
     CHANGE);

  pinMode(AIN1, OUTPUT);
  pinMode(AIN2, OUTPUT);

  // PWM setup
  ledcAttach(PWMA, PWM_FREQ, PWM_RES);
}
```

8.3 Sensor Calibration

Chapter 9

Motion Control

9.1 Differential Drive

9.2 PID Control

Chapter 10

Mapping and Localization

10.1 Maze Representation

10.2 Localization

10.3 Wall Detection

Chapter 11

Path Planning

11.1 Flood Fill

11.2 Shortest Path Optimization

11.3 Alternative Algorithms

Part III

Reference Implementation

Chapter 12

Mechanical Assembly

Chapter 13

Electronics Assembly

Chapter 14

Firmware Configuration

Chapter 15

Testing

Appendix A

Datasheets

Appendix B

Equations

Appendix C

Troubleshooting

Appendix D

Glossary

Contributors

Revision History

- 0.X : Drafts & WIP

References

Installing the ESP32 Board in Arduino IDE